Préparation NCA-AMAD : le cours complet

Cours de préparation à l'examen NVIDIA-Certified Associate : AI Model and Application Development (NCA-AMAD). Rédigé le 2 novembre 2026 d'après les objectifs publiés par NVIDIA sur la page officielle de l'examen.

Avant de commencer

L'examen. 1 heure, 50 à 60 questions à choix multiple, un lab pratique, notation réussite/échec, 125 $. Le sigle exact est NCA-AMAD (et non NCA-MAD).

Poids des domaines. Agentic AI 17 % · Data Processing et Data Flywheel 17 % · Inference 15 % · Personnalisation et fine-tuning 14 % · Models 11 % · AI Enterprise et Blueprints 10 % · Sizing et Configuration 10 % · Physical AI 6 %.

Organisation du cours. Les chapitres suivent un ordre pédagogique, des bases vers le déploiement : 1. Models, 2. Personnalisation et fine-tuning, 3. Agentic AI, 4. Data et data flywheel, 5. Inference, 6. AI Enterprise et Blueprints, 7. Sizing et configuration, 8. Physical AI, 9. Fiche de révision et quiz. Chaque chapitre reprend les sujets officiels, puis se termine par « À retenir » et les pièges fréquents.

Limites. Ce document est une synthèse pédagogique, pas un contenu officiel NVIDIA. Les outils récents (Agent Toolkit, NemoClaw, Switchyard, Dynamo) évoluent vite : je donne leur rôle général et je vous renvoie à la documentation officielle pour les détails. Les chiffres d'exemple sont des ordres de grandeur pour comprendre, pas des valeurs à apprendre par cœur.

Méthode conseillée. Lisez un chapitre, fermez le document, réexpliquez les notions avec vos mots, puis faites les questions du chapitre 9 correspondant au domaine.

1. Models and Model Types (11 %)

Objectif : choisir le bon type de modèle pour un besoin donné et comprendre comment fonctionne un LLM.

1.1 Machine learning classique et réseaux de neurones

- **ML classique** (régression, arbres, forêts aléatoires, gradient boosting) : très efficace sur les données tabulaires, peu coûteux, souvent plus interprétable, mais le feature engineering est manuel.

- **Réseaux de neurones (deep learning)** : apprennent eux-mêmes leurs représentations à partir de données brutes (texte, image, audio). Excellents sur les données non structurées, mais gourmands en données et en calcul.
- **Pourquoi le calcul parallèle ?** L'entraînement et l'inférence reposent sur d'énormes multiplications de matrices. Elles se découpent en calculs indépendants que les milliers de cœurs d'un GPU exécutent simultanément.

1.2 Génératif et discriminatif

- **Discriminatif** : apprend à prédire une étiquette ou à séparer des classes (classer un e-mail, détecter une fraude).
- **Génératif** : apprend la distribution des données pour produire du contenu nouveau (texte, image, audio, vidéo).

1.3 Vision par ordinateur

- **Classification** : une étiquette pour toute l'image.
- **Détection d'objets** : boîtes englobantes avec une classe pour chaque objet.
- **Segmentation** : une étiquette par pixel. La segmentation d'instance distingue en plus chaque objet individuel.
- **OCR** : extraction du texte contenu dans une image.
- **NVIDIA TAO Toolkit** : adapte des modèles de vision pré-entraînés à vos données (transfert d'apprentissage), avec peu d'exemples.

1.4 Modèles non langagiers

- **Embedding** : transforme un contenu (texte, image) en vecteur. Deux contenus proches sémantiquement ont des vecteurs proches. C'est la base de la recherche sémantique et du RAG.
- **Reranking** : reçoit une requête et des passages candidats, puis les re-note finement. Plus lent mais plus précis qu'un embedding, donc appliqué après une première recherche rapide.
- **Multimodal** : traite plusieurs modalités. Un **VLM** (vision-language model) prend image ou vidéo plus texte et produit du texte. Un modèle **omni** combine texte, image, audio et vidéo.
- **World foundation model** : modèle qui prédit ou génère l'évolution d'un monde physique (famille NVIDIA Cosmos, voir chapitre 8).

1.5 Types de modèles de langage

- **LLM** (large language model) : des milliards de paramètres, très polyvalent. **SLM** (small language model) : plus petit, moins cher, plus rapide, déployable en edge. Un SLM est souvent suffisant pour une tâche étroite et pertinent pour les agents.
- **Base ou instruction-tuned** : un modèle de base prédit le token suivant sans suivre forcément les consignes. Un modèle instruct est fine-tuné pour suivre des instructions et dialoguer.
- **Reasoning model** : produit une chaîne de raisonnement avant sa réponse. Meilleur en maths, code et planification, mais consomme plus de tokens, donc plus de latence et de coût.
- **Dense ou MoE** : un modèle dense active tous ses paramètres pour chaque token. Un modèle **mixture-of-experts** contient plusieurs « experts » et un routeur n'en active que quelques-uns par token : calcul par token réduit, mais tous les poids doivent rester en mémoire GPU.
- **Ouvert ou propriétaire** : un modèle à poids ouverts permet le self-hosting, le fine-tuning et le contrôle des données. Une API propriétaire est plus simple à démarrer mais crée une dépendance et fait sortir les données.
- **Foundation ou task-specific** : un modèle fondation est généraliste et adaptable. Un modèle spécifique est entraîné pour une seule tâche.

1.6 Mécanique des LLM

- **Token** : unité de texte (mot, morceau de mot, ponctuation). En anglais, un token vaut environ trois quarts de mot. Coût, latence et limites se comptent en tokens (entrée et sortie).
- **Fenêtre de contexte** : nombre maximal de tokens (prompt plus réponse) traités en une fois. Un contexte long augmente la mémoire et la latence.
- **Génération autorégressive** : le modèle produit un token à la fois, chaque token dépendant de tous les précédents. C'est l'origine de la séparation prefill/decode (chapitre 5).
- **Espace latent / embedding** : espace vectoriel dense où les concepts proches sont proches.
- **Paramètres de génération** : la température et le top-p règlent le degré d'aléatoire. Basse température pour des réponses stables, plus haute pour de la créativité.

1.7 Les modèles NVIDIA : Nemotron

Nemotron est la famille de modèles ouverts de NVIDIA orientée IA agentique (raisonnement, multimodal, RAG, voix, sécurité). Retenez qu'ils existent en plusieurs tailles et sont accessibles via build.nvidia.com et en NIM.

1.8 Critères de sélection d'un modèle

- **Adéquation au workload** : tâche, langue, modalité, fonctions requises (appel d'outils, contexte long).
- **Taille** : elle détermine la mémoire GPU nécessaire.
- **Coût** : par token ou par heure de GPU.
- **Performance** : qualité mesurée sur vos propres données, latence, débit.
- **Contraintes** : licence, confidentialité, déploiement on-premise.

Démarche saine : partir du plus petit modèle qui atteint la qualité visée, évaluer sur un jeu de test applicatif, n'augmenter la taille que si nécessaire.

1.9 Évaluer un modèle

- **Efficacité** : métriques de la tâche (exactitude, F1, taux de réussite) et évaluations sur vos cas réels.
- **Robustesse** : tenue face à des entrées bruitées, reformulées, adversariales ou hors distribution.
- **Interprétabilité** : comprendre pourquoi le modèle répond ainsi. Plus facile avec des modèles simples ; très difficile sur un grand réseau de neurones.

À retenir

- Tabulaire : souvent ML classique. Non structuré : deep learning.
- Embedding = représentation vectorielle. Reranker = re-notation fine des candidats.
- MoE : calcul réduit mais mémoire inchangée.
- Reasoning : meilleur raisonnement, plus de tokens.

Pièges fréquents

- Croire qu'un MoE réduit la mémoire GPU nécessaire : non, seulement le calcul par token.
- Confondre embedding (recherche rapide) et reranker (précision, après la recherche).
- Oublier que les benchmarks généraux ne remplacent pas une évaluation sur vos données.

2. Personnalisation et fine-tuning (14 %)

Objectif : savoir adapter un modèle à un cas d'usage, choisir entre prompt engineering, RAG et fine-tuning, et déployer le résultat.

2.1 Les trois leviers

- **Prompt engineering** : le moins cher et le plus rapide, sans données d'entraînement. Limité par la taille du contexte et la constance des réponses.
- **RAG** (retrieval-augmented generation) : injecte au moment de la requête des connaissances externes, privées ou récentes. Réduit les hallucinations, permet de citer les sources, se met à jour en changeant l'index, sans réentraînement.
- **Fine-tuning** : modifie les poids pour changer le comportement, le style, le format ou le vocabulaire métier, ou pour distiller une capacité dans un modèle plus petit. Demande des données étiquetées de qualité, du calcul et une évaluation.

Règle simple : le RAG apporte du **savoir** (quoi), le fine-tuning façonne le **comportement** (comment). On commence par le prompt, puis le RAG, puis le fine-tuning. Les trois se combinent.

2.2 Prompt engineering et optimisation

- **System prompt** : instructions durables (rôle, ton, règles, format) qui encadrent toutes les requêtes.
- **Zero-shot** : aucune démonstration. **Few-shot** : quelques exemples dans le prompt pour guider le format ou le style.
- **Chain-of-thought** : demander de raisonner étape par étape pour améliorer les tâches de logique.
- **Sortie structurée** : contraindre la réponse à un schéma (JSON) par instructions ou par décodage guidé.
- **Optimisation de prompt** : itérer en mesurant sur un jeu d'évaluation (manuellement ou avec des outils automatiques) plutôt qu'au ressenti.

2.3 Le pipeline RAG

1. **Ingestion** : extraction du texte (y compris tableaux et images des PDF), nettoyage, ajout de métadonnées.
2. **Chunking** : découpage en passages. Trop petits, ils perdent le contexte. Trop grands, ils diluent l'information et coûtent des tokens. On ajuste la taille et le chevauchement, ou on découpe selon la structure du document.
3. **Embeddings** : chaque chunk devient un vecteur.
4. **Indexation vectorielle** : stockage dans une base ou un index de recherche de plus proches voisins approximatifs.
5. **Requête** : la question est vectorisée, on récupère les top-k passages, on les re-note éventuellement avec un reranker, puis on les insère dans le prompt du LLM.
6. **Génération** : le LLM répond en s'appuyant sur ces passages, idéalement avec citations.

Recherche sémantique ou hybride. La recherche sémantique repose uniquement sur les embeddings. La recherche **hybride** combine embeddings et recherche lexicale (type BM25), puis fusionne les résultats. Elle est meilleure sur les termes exacts : acronymes, références produit, codes.

Variantes. RAG sur du code (embeddings adaptés au code), RAG multimodal (extraction des images et tableaux), RAG multi-tours (reformuler la question en tenant compte de l'historique).

Où ça casse. Mauvais chunking, passages pertinents non récupérés, contexte ignoré par le modèle, hallucination malgré le contexte. On mesure la qualité de récupération et le caractère ancré (groundedness) des réponses.

Côté NVIDIA. NeMo Retriever (embedding, reranking, extraction de documents) et le RAG Blueprint.

2.4 Méthodes de fine-tuning

- **SFT** (supervised fine-tuning) : entraînement sur des paires instruction-réponse. En version complète, tous les poids sont mis à jour : coûteux en mémoire.
- **PEFT** (parameter-efficient fine-tuning) : on n'entraîne qu'une petite fraction de paramètres.
  - **LoRA** : on gèle les poids du modèle et on ajoute de petites matrices de faible rang qui apprennent la correction. L'adapter obtenu pèse peu.
  - **QLoRA** : LoRA appliqué à un modèle quantifié (4 bits). Permet d'adapter de gros modèles sur des GPU modestes.
- **Instruction tuning** : SFT sur un large éventail d'instructions pour rendre le modèle apte à suivre des consignes.
- **RLHF** : apprentissage par renforcement à partir de préférences humaines. On entraîne un modèle de récompense, puis on optimise le LLM contre lui.
- **Optimisation de préférences** : **DPO** apprend directement à partir de paires réponse préférée/rejetée, sans modèle de récompense ni boucle de renforcement ; **KTO** n'a besoin que d'un signal binaire bon/mauvais ; **GRPO** compare un groupe de réponses entre elles avec une récompense vérifiable, utilisé pour entraîner les modèles de raisonnement.
- **Distillation** : un petit modèle « élève » apprend des sorties d'un grand « enseignant ». Résultat : un modèle plus rapide et moins cher.

2.5 Données, choix et risques

- **Données** : la qualité prime sur la quantité. Il faut des exemples corrects, variés, au format cohérent, sans données personnelles, avec un jeu de validation séparé.

- **Choisir** : peu de données ou de budget, commencez par prompt ou RAG ; comportement ou format précis, LoRA ; changement profond avec beaucoup de données, SFT complet ; alignement sur des préférences, DPO ; réduction de coût, distillation.
- **Catastrophic forgetting** : le modèle perd des capacités générales en apprenant la tâche. Atténuation : PEFT, mélange de données générales, faible taux d'apprentissage.
- **Overfitting** : il mémorise les exemples d'entraînement au lieu de généraliser. Atténuation : validation, arrêt précoce, plus de données variées.
- **Autres risques** : contamination du jeu de test, biais, dégradation des protections de sécurité.

2.6 Valider avant de déployer

- **Expériences offline** : jeu de test mis de côté, benchmarks généraux pour détecter une régression, évaluation applicative, tests de sécurité.
- **A/B testing** : sur le trafic réel, une part des utilisateurs reçoit le nouveau modèle. On compare les métriques produit (satisfaction, réussite de la tâche, coût). Déploiement progressif (canary) et retour arrière possible.

2.7 Déployer un modèle fine-tuné

- **Adapter serving** : un seul modèle de base reste chargé et plusieurs adapters LoRA sont appliqués à la volée selon la requête. Économique et flexible (NVIDIA NIM sait servir plusieurs adapters LoRA).
- **Model merging** : on fusionne les poids de l'adapter dans le modèle. On obtient un modèle unique sans surcoût d'inférence, mais moins flexible : un déploiement par variante.

À retenir

- RAG pour la connaissance fraîche ou privée, fine-tuning pour le comportement.
- LoRA gèle le modèle et entraîne de petites matrices ; QLoRA ajoute la quantification.
- DPO n'a pas de modèle de récompense ; GRPO sert au raisonnement.
- Valider offline puis en A/B avant le déploiement complet.

Pièges fréquents

- Proposer du fine-tuning pour injecter des faits qui changent souvent : le RAG est mieux adapté.
- Confondre distillation (petit modèle qui imite un grand) et quantification (réduction de précision).
- Oublier le risque de forgetting et le jeu de validation séparé.

3. Agentic AI (17 %)

Objectif : comprendre comment un agent raisonne, utilise des outils, s'orchestre, reste sûr et se mesure.

3.1 Qu'est-ce qu'un agent ?

Un agent est un système construit autour d'un LLM qui poursuit un objectif en boucle : il observe, planifie, appelle des outils, agit, puis ajuste, avec une autonomie encadrée.

- **Face à une application générative classique** (une requête, une réponse, aucune action externe), l'agent décide dynamiquement des étapes et agit sur des systèmes externes.
- **Face à un système à règles**, qui est déterministe et codé à l'avance, l'agent gère l'imprévu mais devient non déterministe.

Briques de base : le modèle (le « cerveau »), les instructions, les outils, la mémoire, la planification et l'orchestration, les garde-fous. Le **harness** (ou runtime) est la couche logicielle qui exécute la boucle : il appelle le modèle, exécute les outils, gère l'état, les erreurs et les limites.

Appel LLM unique ou workflow multi-étapes ? Un appel unique suffit pour une tâche simple et bien définie. Un workflow multi-étapes (chaînage, routage, parallélisation) convient quand la tâche se décompose. Un agent autonome se justifie quand les étapes ne sont pas connues à l'avance. Chaque étape ajoute latence, coût et risque d'erreur cumulée : on commence simple.

3.2 Patterns de raisonnement

- **ReAct** (reason + act) : boucle pensée → action (appel d'outil) → observation → nouvelle pensée, jusqu'à la réponse.
- **Reflection** : l'agent critique sa propre sortie et la corrige.
- **Planner-executor** : un planificateur décompose l'objectif en étapes, des exécuteurs les réalisent, avec replanification si besoin. Adapté aux tâches longues, au prix d'un surcoût de planification.

3.3 Outils et intégrations

- **Tool use / function calling** : le modèle émet un appel structuré (nom de l'outil, arguments en JSON). Le runtime exécute l'outil et renvoie le résultat au modèle. Le modèle n'exécute rien lui-même.
- **APIs et systèmes externes** : bases de données, moteurs de recherche, exécution de code, applications métier.

- **MCP** (Model Context Protocol) : standard ouvert pour exposer outils et données à des agents via un serveur et un client, ce qui évite les intégrations sur mesure. NeMo Agent Toolkit peut jouer le rôle de client MCP.

3.4 Orchestration

- **Ordre d'exécution** : séquentiel, parallèle (appels indépendants), conditionnel (routage), en boucle. Il dépend des dépendances de données entre outils.
- **Robustesse** : délais d'attente, nouvelles tentatives, limite d'itérations, gestion d'erreurs.
- **Routage entre modèles** : envoyer chaque requête vers le modèle adapté (petit pour le simple, grand pour le complexe). NeMo Switchyard est présenté par NVIDIA pour router des charges d'agents entre modèles.

3.5 État, contexte et mémoire

- **Stateless** : chaque appel est indépendant. Simple et facile à mettre à l'échelle. **Stateful** : l'état de la conversation ou de la tâche est conservé.
- **Context engineering** : choisir avec soin ce qui entre dans la fenêtre de contexte (instructions, outils utiles, résultats, historique résumé) pour maximiser la qualité dans le budget de tokens.
- **Mémoire court terme** : le contexte de la session en cours, utile pour la tâche présente.
- **Mémoire long terme** : stockée hors du modèle (base vectorielle, profil, historique), réutilisable entre sessions. Utile pour la personnalisation et les connaissances persistantes, avec des enjeux de confidentialité et d'obsolescence.

3.6 Systèmes multi-agents

- **Composants** : agents spécialisés, orchestrateur ou superviseur, protocole de communication, mémoire partagée.
- **Patterns de collaboration** : superviseur et exécutants, pipeline séquentiel, critique mutuelle, organisation hiérarchique.
- **Single ou multi ?** Un seul agent suffit pour une tâche cohérente avec peu d'outils. Le multi-agent aide quand les domaines sont distincts, qu'il y a trop d'outils pour un seul prompt, qu'on veut du parallélisme ou séparer les permissions. Contreparties : plus de tokens, de latence, de complexité et un débogage plus dur.

3.7 Sécurité et garde-fous

Risques et modes de défaillance : hallucinations, mauvais outil ou mauvais arguments, boucles infinies, **prompt injection** (directe ou indirecte via un contenu récupéré), fuite de données, actions destructrices, erreurs qui s'accumulent sur plusieurs étapes.

Contrôles :

- **Guardrails** (NeMo Guardrails) : règles sur les entrées, les sorties, le dialogue, la récupération et l'exécution.
- **Filtres de sortie** : contenu toxique, données sensibles, format.
- **Défenses contre l'injection** : séparer instructions et données non fiables, moindre privilège, validation des arguments d'outils, listes d'actions autorisées, exécution en bac à sable.
- **Réduction des hallucinations** : ancrage dans des sources (RAG), vérification des faits, citations.
- **Tracing** : tracer chaque étape pour comprendre et auditer.
- **Human-in-the-loop** : validation humaine avant les actions à risque (paiement, suppression, envoi), possibilité d'interrompre ou d'escalader.

NemoClaw et OpenShell sont des outils NVIDIA liés à l'exécution sécurisée d'agents (le cours DLI gratuit « Securing Agents with NemoClaw and OpenShell » y est consacré). Retenez leur rôle de sécurisation et consultez la documentation pour le détail.

3.8 Mesurer et améliorer

- **Métriques** : taux de réussite de la tâche, exactitude, tokens consommés, latence, coût, nombre d'étapes et d'appels d'outils, taux d'erreur des outils. Un workflow multi-étapes multiplie la consommation de ressources.
- **Améliorer** : collecter du feedback, choisir un meilleur modèle (ou un plus petit par distillation), affiner prompts, descriptions d'outils et workflow, et évaluer sur un jeu de scénarios avant chaque changement.
- **Outils NVIDIA** : NeMo (microservices), NeMo Agent Toolkit (construire, connecter, évaluer et profiler des agents), NeMo Guardrails, NemoClaw, modèles Nemotron.

À retenir

- Agent = boucle autonome avec outils, plus flexible mais moins prévisible qu'un système à règles.
- ReAct = penser, agir, observer. Planner-executor = planifier puis exécuter.
- Le modèle propose l'appel d'outil, le runtime l'exécute.
- Guardrails, moindre privilège et validation humaine contre les risques.

Pièges fréquents

- Choisir un agent multi-étapes pour une tâche qu'un seul appel résout.
- Penser que le modèle exécute lui-même les outils.
- Oublier que l'injection de prompt peut venir d'un document récupéré, pas seulement de l'utilisateur.

4. Data Processing et Data Flywheel (17 %)

Objectif : transformer des données brutes en données fiables, les gouverner, évaluer un système, et faire de la production une source d'amélioration continue.

4.1 Types de données

- **Structurées** : tables, bases SQL.
- **Semi-structurées** : JSON, XML, journaux (logs).
- **Non structurées** : texte, images, audio, PDF. C'est la majorité des données d'entreprise, d'où l'importance des étapes d'extraction.

4.2 Ingestion et ETL

- **ETL** : extraire, transformer, charger. La variante ELT charge d'abord, transforme ensuite dans l'entrepôt de données.
- **Batch ou streaming ?** Le batch traite de gros volumes par lots périodiques : fort débit, latence élevée (réindexation nocturne, préparation d'un jeu d'entraînement). Le streaming traite un flux continu avec une latence faible (détection de fraude en temps réel).
- **Outils** : Apache Spark (traitement distribué), Databricks (plateforme de données), Kafka (flux d'événements), NVIDIA RAPIDS (bibliothèques GPU pour accélérer pandas, le machine learning classique et Spark, souvent sans réécrire le code).
- **Trois flux à distinguer** : le flux d'entraînement (collecte, nettoyage, entraînement), le flux d'inférence (requête, prétraitement, modèle, post-traitement) et le flux de retrieval (documents, chunks, embeddings, index, requête).

4.3 Qualité et préparation des données

Problèmes de qualité classiques :

- **Valeurs manquantes** : imputation ou suppression.
- **Doublons** : exacts ou quasi-doublons, à dédupliquer.
- **Données périmées** (staleness) : l'information n'est plus à jour.

- **Incohérences de format**, valeurs aberrantes, étiquettes bruitées, classes déséquilibrées.

Prétraitement du texte : normalisation, suppression du HTML et du bruit, détection de langue, filtrage de qualité, déduplication, retrait des données personnelles (PII) et des contenus toxiques ou nuisibles. Pour ce travail à grande échelle sur GPU, NVIDIA propose par exemple NeMo Curator.

Étiquetage (labeling) : manuel, par des experts, semi-automatique ou assisté par LLM, avec un contrôle qualité (accord entre annotateurs).

Feature engineering et feature store :

- Le **feature engineering** crée ou transforme des variables utiles au modèle (encodage de catégories, normalisation, agrégats temporels).
- Le **feature store** est un dépôt centralisé et versionné de features réutilisables. Il garantit la cohérence entre entraînement et production et évite le décalage appelé training-serving skew.
- L'**analyse exploratoire (EDA)** inspecte distributions, corrélations et anomalies avant de modéliser.

Impact sur la performance : la préparation des données (qualité, fraîcheur, fiabilité) influence la performance du modèle autant que le modèle lui-même. Des données médiocres donnent un système médiocre.

4.4 Gouvernance, traçabilité et confidentialité

- **Gouvernance des données** : politiques de propriété, d'accès, de qualité et de conformité.
- **Lignage (lineage)** : d'où vient une donnée et quelles transformations elle a subies. **Provenance** : origine et droits d'usage. Ensemble, ils permettent l'audit, la reproductibilité et le débogage.
- **Model risk management** : identifier, mesurer et réduire les risques liés aux modèles (biais, dérive, erreurs) par la documentation, la validation et le suivi.
- **Confidentialité** : détecter et masquer ou anonymiser les PII ; limiter la **durée de conservation** (data retention) ; respecter la **souveraineté des données** (les données relevent du droit du pays où elles sont stockées, d'où l'intérêt du self-hosting) ; surveiller les **biais** (échantillons non représentatifs) et les **risques de détournement** (misuse).

4.5 Évaluer : métriques et jeux de données

- Les **benchmarks généraux** comparent des modèles sur des tâches standard, mais reflètent mal votre cas d'usage.

- Une **évaluation applicative**, basée sur un jeu de test construit à partir de vos vrais cas, prédit mieux la performance en production. C'est pourquoi elle est souvent plus utile qu'un benchmark général.
- **Métriques courantes** : exactitude, groundedness (la réponse est-elle ancrée dans les sources ?), taux d'hallucination, réussite de la tâche, pertinence. L'usage d'un LLM comme juge est fréquent mais doit être validé par un échantillon humain. NeMo Evaluator aide à automatiser ces évaluations.

4.6 Le data flywheel

Le **data flywheel** est une boucle d'amélioration continue :

1. On déploie le système.
2. On collecte des signaux d'usage : feedback explicite (pouce haut ou bas) et implicite (reformulations, abandons, corrections), plus les journaux.
3. On filtre, nettoie et étiquette ces données.
4. On améliore : fine-tuning, distillation, réglage du RAG ou des prompts.
5. On évalue puis on redéploie. Un meilleur système attire plus d'usage, donc plus de données.

Le **Data Flywheel Blueprint** de NVIDIA illustre l'idée : exploiter les données de production d'un agent pour distiller ou fine-tuner des modèles plus petits qui conservent la qualité à moindre coût.

Défis : qualité et biais du feedback, PII présentes dans les journaux, boucle de rétroaction qui renforce ses propres erreurs, coût d'annotation, nécessité d'évaluer avant de redéployer, gouvernance.

Surveillance en production : exactitude, groundedness, taux d'hallucination, réussite de la tâche, latence et coût.

À retenir

- Batch = débit, streaming = faible latence.
- Lignage et provenance servent à l'audit ; PII, retention et souveraineté encadrent la confidentialité.
- Une évaluation sur vos propres cas vaut mieux qu'un benchmark général.
- Flywheel = déployer, collecter, curer, améliorer, évaluer, redéployer.

Pièges fréquents

- Confondre lignage (parcours des données) et provenance (origine et droits).
- Réinjecter les données de production brutes dans l'entraînement sans filtrage ni retrait des PII.

- Ignorer le décalage entre features d'entraînement et de production.

5. Inference Concepts (15 %)

Objectif : comprendre comment un LLM est servi, comment on mesure et améliore sa performance, et ce qui fait varier le coût.

5.1 Les phases de l'inférence

- **Prefill** : le prompt entier est traité en parallèle et le KV cache est construit. Phase limitée par le calcul. Elle détermine le **time to first token (TTFT)**.
- **Decode** : un token est généré à la fois, en relisant les poids et le KV cache à chaque pas. Phase limitée par la bande passante mémoire. Elle détermine la latence entre tokens (**ITL** ou TPOT).
- **Modes de service** : *temps réel* (interactif, faible latence, petits lots), *batch* ou offline (débit maximal, latence secondaire, par exemple résumer un million de documents), *streaming* (les tokens sont renvoyés au fil de l'eau, ce qui réduit la latence perçue).

5.2 Mesurer la performance

- **Latence** : TTFT, ITL, latence de bout en bout (environ TTFT plus nombre de tokens fois ITL).
- **Débit** : tokens par seconde (par utilisateur et au total), requêtes par seconde.
- **Percentiles** (p50, p95, p99) : la queue de distribution compte pour l'expérience réelle.
- **Load testing** : simuler une charge réaliste (concurrence, longueurs d'entrée et de sortie) pour trouver la capacité et vérifier les engagements de latence.
- **Compromis latence/débit** : regrouper plus de requêtes dans un lot augmente le débit total mais allonge la latence de chaque requête.

5.3 Techniques d'optimisation

- **Quantization** : réduire la précision des poids (et parfois des activations) de FP16 vers FP8, INT8 ou 4 bits. Gains : moins de mémoire, calcul plus rapide. Risque : perte de précision, à mesurer sur vos évaluations.
- **KV cache** : conserve les clés et valeurs d'attention déjà calculées pour ne pas les recalculer à chaque token. Il croît avec la longueur du contexte et le nombre de requêtes simultanées, et devient un poste majeur de mémoire.
- **Prefix caching / KV cache reuse** : réutiliser le cache d'un préfixe commun (system prompt, document partagé) pour réduire le prefill et le TTFT.

- **Paged attention** : gère le KV cache par pages, comme une mémoire virtuelle. Moins de fragmentation, donc plus de requêtes en parallèle.
- **Batching** : le batching statique attend un lot complet. Le **continuous batching** (in-flight batching dans TensorRT-LLM) insère de nouvelles requêtes dès qu'une autre se termine, ce qui améliore l'utilisation du GPU. Le **dynamic batching** (Triton) regroupe les requêtes arrivant dans une courte fenêtre.
- **Speculative decoding** : un petit modèle « draft » propose plusieurs tokens, le grand modèle les vérifie d'un coup. Latence réduite sans changer la qualité de sortie, surtout quand le draft est souvent accepté.
- **Parallélisme** : le *tensor parallelism* découpe les matrices d'une couche entre plusieurs GPU (permet les gros modèles, réduit la latence, exige une interconnexion rapide comme NVLink) ; le *pipeline parallelism* répartit les couches en étages ; l'*expert parallelism* répartit les experts d'un MoE ; la réplication (data parallelism) multiplie le débit.
- **Disaggregated serving** : séparer prefill et decode sur des groupes de GPU différents, chacun optimisé pour sa contrainte (calcul ou bande passante). C'est l'une des idées de NVIDIA Dynamo.
- **Décodage guidé** : contraindre la sortie à un schéma (JSON) pour des réponses structurées fiables.
- **Effet de la longueur de contexte** : le KV cache croît linéairement avec le contexte et le nombre de requêtes. Un contexte long limite donc la concurrence. Si le GPU manque de mémoire (erreur out-of-memory), on réduit la longueur maximale ou la taille de lot, on quantifie, ou on ajoute des GPU.

5.4 Infrastructure de service

- **Serveur de modèle** : charge le modèle, expose une API et gère le batching. Exemples côté NVIDIA : NIM (conteneur prêt à l'emploi), Triton Inference Server, TensorRT-LLM (bibliothèque d'optimisation).
- **Gateway et routage** : point d'entrée unique qui gère authentification, quotas, répartition de charge, routage vers le bon modèle ou réplica, bascule en cas de panne. La **provider abstraction** offre une interface unifiée : on change de modèle ou de fournisseur sans toucher à l'application.
- **NVIDIA Dynamo** : framework d'inférence distribuée pour de nombreux GPU, avec service disagrégé et routage tenant compte du KV cache. Consultez la documentation pour le détail des fonctions.
- **Autoscaling** : ajuster le nombre de réplicas à la charge (file d'attente, utilisation GPU, latence, requêtes concurrentes).

- **Request queuing** : quand le système est saturé, les requêtes attendent, ce qui augmente la latence. On fixe des limites et une contre-pression.
- **Cold start** : un nouveau réplica met du temps à démarrer (chargement de poids volumineux). Remèdes : cache des modèles, nombre minimal de réplicas toujours actifs, préchauffage. Descendre à zéro réplica économise de l'argent mais pénalise le premier utilisateur.
- **Stratégies de scaling** : horizontal (plus de réplicas) ou vertical (plus de GPU ou GPU plus gros par instance).

5.5 Coûts d'inférence

- **Facteurs** : taille du modèle, nombre de tokens (la sortie coûte généralement plus que l'entrée), volume de trafic, matériel (type, nombre, durée), cibles de latence (plus strictes, donc moins de batching et GPU moins bien utilisés), redondance pour la disponibilité.
- **Leviers** : modèle plus petit ou quantifié, caching, batching, routage vers un petit modèle quand c'est possible, prompts plus courts, limite de tokens en sortie.
- **Ordre de grandeur** : coût par token ≈ coût du GPU par heure divisé par le nombre de tokens produits par heure. Mieux on utilise le GPU, moins le token coûte.

À retenir

- Prefill = calcul, produit le TTFT. Decode = bande passante mémoire, produit l'ITL.
- Le KV cache accélère mais consomme la mémoire ; le prefix caching réduit le TTFT.
- Continuous batching améliore le débit ; speculative decoding réduit la latence.
- Latence et débit se contrebalancent via la taille de lot.

Pièges fréquents

- Confondre TTFT (prefill) et latence entre tokens (decode).
- Penser que quantifier est gratuit : il faut réévaluer la qualité.
- Oublier que le contexte long réduit le nombre de requêtes simultanées.
- Confondre tensor parallelism (découpe d'une couche) et pipeline parallelism (découpe en étages de couches).

6. NVIDIA AI Enterprise et Blueprints (10 %)

Objectif : comprendre comment passer d'un prototype à une application d'entreprise en production avec la pile logicielle NVIDIA.

6.1 L'AI Factory

Une **AI Factory** est une infrastructure complète qui produit de l'intelligence à grande échelle. On la lit en couches :

- **Infrastructure** : GPU, CPU, réseau, stockage.
- **Plateforme** : orchestration (Kubernetes), gestion des ressources (par exemple Run:ai), exploitation, logiciels d'entreprise comme NVIDIA AI Enterprise.
- **Modèles** : modèles ouverts ou sur mesure (par exemple Nemotron, Cosmos).
- **Applications** : agents, assistants, Blueprints.

Les **profils de workload** (entraînement, fine-tuning, inférence, simulation) ont des besoins différents en calcul, mémoire, réseau et stockage ; la pile doit les accueillir.

6.2 NVIDIA AI Enterprise

- **Idée** : plateforme logicielle cloud-native et supportée, qui regroupe bibliothèques, frameworks et microservices (dont NIM et NeMo) pour passer du prototype à la production.
- **Problèmes résolus** : sécurité et correctifs, stabilité des versions, support, performance, conformité, plutôt que d'assembler soi-même des composants open source.
- **Environnements** : on-premise, cloud, edge, hybride, avec le même logiciel partout.

6.3 NVIDIA NIM

- **Ce que c'est** : un microservice conteneurisé qui empaquette un modèle, un moteur d'inférence optimisé, ses dépendances et une API standard (compatible OpenAI pour les LLM).
- **Sélection selon le calcul** : le NIM choisit un profil optimisé pour le GPU détecté.
- **Différence avec un serving traditionnel** : pas de conversion ni d'optimisation manuelle à faire, le packaging est prêt à déployer (une commande de conteneur, ou des charts sur Kubernetes).
- **Self-hosting** : on garde le contrôle des données (y compris en environnement isolé, air-gapped).

6.4 Blueprints

Les **Blueprints** sont des **architectures de référence** : workflows complets (code, documentation, déploiement) construits sur NIM et NeMo, qui accélèrent la mise en œuvre.

- Assistant sur documents internes : **RAG Blueprint**.

- Améliorer des agents avec des données de production : **Data Flywheel Blueprint**.
- Détection de transactions frauduleuses : **blueprint de détection de fraude**.

Pour associer un blueprint à un scénario, partez du besoin métier, pas de la technique.

6.5 Déploiement et orchestration

- **Conteneurs** (Docker) : empaquettent application et dépendances pour un comportement identique partout.
- **Kubernetes** : orchestre les conteneurs (planification sur GPU, réplicas, mises à jour progressives, autoscaling).
- **Infrastructure as code** : l'infrastructure et les déploiements sont décrits dans des fichiers versionnés (par exemple Terraform, Helm), donc reproductibles.
- **Service mesh** : couche qui gère la communication entre microservices (chiffrement, routage du trafic, nouvelles tentatives, observabilité).
- **NIM Operator** : gère le cycle de vie des NIM sur Kubernetes (déploiement, mise en cache des modèles, mise à l'échelle).

6.6 Cas d'usage de productivité

Assistants et copilotes, questions-réponses sur les documents d'entreprise (RAG), résumés, génération de code, support client, extraction d'informations.

À retenir

- AI Enterprise = pile logicielle supportée pour la production, déployable partout.
- NIM = modèle + moteur optimisé + API standard dans un conteneur.
- Blueprint = architecture de référence prête à adapter.
- Kubernetes orchestre, l'IaC rend reproductible, le service mesh gère les communications.

Pièges fréquents

- Confondre NIM (un microservice d'inférence) et Blueprint (un workflow complet de référence).
- Oublier que le self-hosting répond à des exigences de contrôle des données.
- Voir Kubernetes comme un outil d'entraînement : il orchestre le déploiement des services.

7. Sizing et Configuration (10 %)

Objectif : dimensionner et configurer un système d'IA, le suivre dans le temps et le faire évoluer.

7.1 Cycle de vie d'un modèle

- **Étapes** : données, entraînement ou fine-tuning, évaluation, déploiement, monitoring, réentraînement ou retrait.
- **Versioning** : versionner modèles, adapters, données, prompts et configurations (avec un registre de modèles) pour reproduire un résultat et revenir en arrière.
- **Release management** : déploiement progressif (canary, blue/green), validation avant généralisation, retour arrière (rollback).
- **Déclencheurs de réentraînement** : dérive détectée, baisse des métriques, nouvelles données significatives, changement de besoin métier, calendrier régulier, incident.

7.2 Monitoring et métriques

- **Observabilité** : comprendre l'état interne d'un système à partir de ce qu'il émet : métriques, journaux (logs) et traces.
- **Drift** : le *data drift* est un changement de distribution des entrées ; le *concept drift* est un changement de la relation entre entrées et sorties. On les détecte en comparant statistiquement la production à une période de référence. Le *post-training drift* désigne la dégradation après l'entraînement.
- **Indicateurs de santé** : disponibilité, taux d'erreur, latence, débit, utilisation GPU et mémoire.
- **SLA et SLO** : engagements de niveau de service. On détecte leur dépassement par des alertes.
- **MTTD** (mean time to detect) : temps moyen pour détecter un incident. **MTTR** (mean time to resolve ou recover) : temps moyen pour le résoudre ou rétablir le service.

7.3 Dimensionner le calcul

Mémoire GPU ≈ poids du modèle + KV cache + activations et surcoûts.

- **Poids** = nombre de paramètres × octets par paramètre. FP16 ou BF16 : 2 octets. FP8 ou INT8 : 1 octet. 4 bits : 0,5 octet. Exemples : 7 milliards en FP16 ≈ 14 Go ; 70 milliards en FP16 ≈ 140 Go (donc plusieurs GPU), en FP8 ≈ 70 Go.
- **KV cache** : croît avec la longueur de contexte et le nombre de requêtes simultanées. Il s'ajoute aux poids.
- **Réplicas** : instances identiques pour augmenter le débit et assurer la haute disponibilité.
- **Scaling horizontal** : ajouter des instances. Élastique, tolérant aux pannes, adapté aux services sans état, mais demande un répartiteur de charge. **Scaling vertical** : donner plus de GPU ou un GPU plus gros à une instance. Nécessaire quand le modèle ne tient pas dans un seul GPU, limité par le matériel disponible.

Démarche de dimensionnement :

1. Estimer les utilisateurs concurrents, les tokens moyens en entrée et en sortie, le débit voulu.
2. Fixer les objectifs de latence (TTFT, ITL) et une marge pour les pics.
3. Choisir modèle et précision, puis vérifier que poids et KV cache tiennent en mémoire.
4. Mesurer par un test de charge.
5. En déduire le nombre de réplicas et le réglage de l'autoscaling.

7.4 Paramètres de configuration qui influencent la performance

- **Taille de lot (batch size)** : plus grande, plus de débit et plus de mémoire, mais plus de latence par requête.
- **Longueur maximale de séquence** : plus longue, plus de contexte possible mais un KV cache plus gros et moins de requêtes simultanées.
- **Précision** : plus basse (FP8, 4 bits), moins de mémoire et plus de vitesse, avec un risque de perte de qualité.
- **Utilisation matérielle et coût** : viser un GPU bien utilisé sans dépasser les cibles de latence.
- **NVIDIA Run:ai** : gestion et planification des GPU sur Kubernetes (partage de GPU, quotas, priorités, mutualisation) pour mieux utiliser les ressources et réduire le coût.

7.5 Observabilité des systèmes déployés

- **Tableaux de bord** pour voir l'état en un coup d'œil.
- **Tracing** des prompts, des étapes d'agent et de la récupération (RAG) : quel outil, quel passage, combien de tokens, quelle latence.
- **Journalisation et alertes**, **télémétrie** (collecte continue de mesures).
- **Détection de drift** et **détection d'anomalies** (par exemple sur des séries temporelles de métriques).

7.6 Opérations d'une AI Factory

- **Intégration de la chaîne d'outils** : CI/CD, supervision, orchestration et registres reliés entre eux.
- **Runbook** : procédure pas à pas pour traiter un incident connu.
- **Playbook** : ensemble de réponses prédéfinies pour un scénario plus large, souvent automatisable.

À retenir

- Poids = paramètres × octets ; ajouter le KV cache.

- Horizontal = plus d'instances ; vertical = plus de GPU par instance.
- MTTD = détecter, MTTR = résoudre.
- Batch size et longueur de séquence font varier débit, latence et mémoire.

Pièges fréquents

- Dimensionner uniquement sur les poids en oubliant le KV cache.
- Confondre data drift (entrées) et concept drift (relation entrée-sortie).
- Confondre runbook (un incident précis) et playbook (scénario plus large).

8. Physical AI (6 %)

Objectif : situer l'IA physique par rapport à l'IA numérique, et connaître simulation, données synthétiques et modèles NVIDIA associés.

8.1 Physical AI et digital AI

La **Physical AI** concerne les systèmes qui perçoivent et agissent dans le monde physique : robots, véhicules autonomes, caméras intelligentes. Par rapport à l'IA purement numérique, elle impose du temps réel, des exigences de sûreté, des capteurs, de l'inférence embarquée et un coût d'erreur plus élevé.

8.2 Le pattern des trois ordinateurs

1. **Ordinateur d'entraînement** (type DGX) : entraîner et fine-tuner les modèles.
2. **Ordinateur de simulation** (Omniverse, Isaac Sim, Cosmos) : simuler, tester, générer des données synthétiques.
3. **Ordinateur embarqué** (type Jetson ou DRIVE AGX) : inférence en temps réel dans le robot ou le véhicule.

Edge inference : on exécute le modèle au plus près de la machine pour une faible latence, une indépendance vis-à-vis du réseau et de la confidentialité.

8.3 Simulation et jumeaux numériques

- **Jumeau numérique (digital twin)** : réplique virtuelle fidèle d'un environnement, d'une usine ou d'un robot.
- **NVIDIA Omniverse** : plateforme de simulation et de jumeaux numériques basée sur OpenUSD.
- **Isaac Sim** : simulateur de robotique construit sur Omniverse, avec capteurs et physique.

- **Données synthétiques** : générées par simulation (par exemple avec Replicator), étiquetées automatiquement, utiles pour les cas rares ou dangereux et pour passer à l'échelle.
- **Domain randomization** : faire varier au hasard textures, éclairages, positions, paramètres physiques pour que le modèle généralise au monde réel.
- **Sim-to-real gap** : écart entre simulation et réalité (physique approximative, rendu, capteurs) qui dégrade la performance une fois déployé. On le réduit par la randomisation, une simulation plus fidèle, des données réelles mélangées et du fine-tuning.
- **Sim-to-real flywheel** : boucle simuler, entraîner, déployer, collecter les données et échecs réels, puis les réinjecter dans la simulation et l'entraînement.

8.4 Familles de modèles NVIDIA

- **Cosmos** : famille de *world foundation models* pour la Physical AI (génération et prédiction de scènes physiquement plausibles, augmentation de données synthétiques, raisonnement sur le monde physique).
- **Isaac GR00T** : modèles fondation pour robots humanoïdes (perception et instruction en entrée, actions en sortie).

À retenir

- Trois ordinateurs : entraînement, simulation, embarqué.
- Données synthétiques + domain randomization réduisent le sim-to-real gap.
- Cosmos = world foundation models ; GR00T = modèles pour humanoïdes.

Piège fréquent

- Confondre Omniverse (plateforme de simulation), Isaac Sim (simulateur robotique), Cosmos (modèles du monde) et GR00T (modèles de robot).

9. Fiche de révision et quiz

9.1 Comparatifs à connaître

- **Prompt, RAG ou fine-tuning** : prompt = rapide, sans données ; RAG = savoir récent ou privé ; fine-tuning = comportement, style, format.
- **Embedding ou reranker** : recherche rapide ou re-notation fine des candidats.
- **LoRA, QLoRA, SFT complet** : adapters légers ; adapters sur modèle 4 bits ; tous les poids mis à jour.
- **RLHF, DPO, KTO, GRPO** : renforcement avec modèle de récompense ; préférences par paires sans renforcement ; signal bon/mauvais ; récompense vérifiable en groupe.

- **Adapter serving ou merging** : plusieurs adapters sur un modèle de base, ou poids fusionnés en un seul modèle.
- **Single ou multi-agent** : simple et cohérent ou domaines distincts, outils nombreux, parallélisme.
- **Stateless ou stateful** ; **mémoire court ou long terme** : sans état ou avec état ; contexte de la session ou stockage persistant.
- **Batch ou streaming** : débit et latence élevée, ou flux et faible latence.
- **Prefill ou decode** : calcul et TTFT, ou bande passante mémoire et latence entre tokens.
- **Tensor, pipeline, expert parallelism** : découpe d'une couche ; étages de couches ; experts d'un MoE.
- **NIM ou Blueprint** : microservice d'inférence ou architecture de référence complète.
- **Horizontal ou vertical** : plus d'instances ou plus de GPU par instance.
- **Lignage ou provenance** : parcours des transformations ou origine et droits.
- **Data drift ou concept drift** : entrées qui changent ou relation entrée-sortie qui change.

9.2 Pièges transversaux

- Les questions « quand utiliser X plutôt que Y » se résolvent presque toujours par coût, données disponibles, fraîcheur de l'information et latence.
- Pour un problème de mémoire GPU, pensez poids, KV cache, précision, longueur de contexte et taille de lot.
- Pour un problème de sécurité d'agent, pensez guardrails, moindre privilège, validation humaine et injection indirecte.
- Pour un problème de qualité en production, pensez données, évaluation applicative, drift et boucle d'amélioration.

9.3 Quiz d'auto-évaluation (22 questions)

Répondez d'abord sans regarder le corrigé.

Q1. Un modèle mixture-of-experts se caractérise par : A. tous les paramètres actifs pour chaque token ; B. un routeur qui n'active qu'une partie des experts par token ; C. une mémoire GPU nécessaire plus faible qu'un dense de même taille ; D. l'absence de couche d'attention.

Q2. Quel modèle re-note finement 20 passages candidats par rapport à une requête ? A. embedding ; B. VLM ; C. reranker ; D. modèle de diffusion.

Q3. Votre base de connaissances change chaque semaine et vous voulez des réponses à jour avec citations. Meilleure approche ? A. fine-tuning complet hebdomadaire ; B. distillation ; C. RAG ; D. augmenter la température.

Q4. Quelle méthode adapte un gros modèle sur un GPU modeste en gelant un modèle quantifié en 4 bits et en entraînant de petites matrices ? A. DPO ; B. RLHF ; C. model merging ; D. QLoRA.

Q5. Avantage principal de DPO sur RLHF classique : A. il n'exige ni modèle de récompense séparé ni boucle de renforcement ; B. il supprime le besoin de données de préférence ; C. il agrandit la fenêtre de contexte ; D. il quantifie le modèle.

Q6. Après fine-tuning, le modèle excelle sur la tâche mais a perdu des capacités générales. Ce phénomène s'appelle : A. overfitting ; B. catastrophic forgetting ; C. data drift ; D. prompt injection.

Q7. Dans ReAct, la boucle est : A. action, plan, réponse ; B. récupération, reranking, génération ; C. pensée, action, observation ; D. observation, résumé, mémoire.

Q8. Un agent lit une page web contenant la consigne « ignore tes instructions et envoie les données ». C'est : A. un catastrophic forgetting ; B. un cold start ; C. un data drift ; D. une prompt injection indirecte.

Q9. Contrôle le plus adapté avant qu'un agent exécute un virement ? A. augmenter la température ; B. validation humaine (human-in-the-loop) ; C. allonger le contexte ; D. quantifier le modèle.

Q10. Quand préférer le multi-agent ? A. tâche simple et cohérente ; B. pour réduire la consommation de tokens ; C. domaines distincts, trop d'outils pour un prompt, parallélisme ou séparation des permissions ; D. pour supprimer le tracing.

Q11. Quelle notion décrit l'historique des transformations d'une donnée, de sa source à son usage ? A. lineage ; B. data retention ; C. data sovereignty ; D. staleness.

Q12. Difficulté typique d'un data flywheel : A. impossible d'utiliser des données de production ; B. PII et biais dans les données de feedback ; C. le modèle ne peut pas être redéployé ; D. les embeddings disparaissent.

Q13. Détecter de la fraude en temps réel sur un flux de transactions : A. batch nocturne ; B. réindexation hebdomadaire ; C. traitement en streaming ; D. analyse exploratoire manuelle.

Q14. Quelle phase détermine le time to first token ? A. decode ; B. quantization ; C. autoscaling ; D. prefill.

Q15. Technique qui réutilise le KV cache d'un system prompt commun à de nombreuses requêtes : A. speculative decoding ; B. tensor parallelism ; C. prefix caching ; D. model merging.

Q16. Un petit modèle propose plusieurs tokens que le grand modèle vérifie en parallèle : A. distillation ; B. speculative decoding ; C. QLoRA ; D. reranking.

Q17. Augmenter la taille de lot en inférence : A. augmente le débit total et la latence par requête ; B. réduit à la fois le débit et la latence ; C. n'a aucun effet ; D. réduit la mémoire nécessaire.

Q18. NVIDIA NIM est : A. un outil d'annotation de données ; B. un conteneur qui empaquette un modèle, un moteur d'inférence optimisé et une API ; C. un type de GPU ; D. un langage de requête.

Q19. Ordre de grandeur des poids d'un modèle de 70 milliards de paramètres en FP16 : A. 35 Go ; B. 70 Go ; C. 140 Go ; D. 280 Go.

Q20. Le MTTR mesure : A. le temps moyen de détection ; B. le temps moyen de résolution ou de rétablissement ; C. le débit moyen ; D. la latence médiane.

Q21. Le pattern des trois ordinateurs NVIDIA pour la Physical AI : A. entraînement, simulation, embarqué ; B. CPU, GPU, DPU ; C. client, serveur, base de données ; D. développement, test, production.

Q22. Pour réduire le sim-to-real gap : A. augmenter la taille du modèle seul ; B. randomisation de domaine, simulation plus fidèle et données réelles mélangées ; C. supprimer la simulation ; D. quantifier le modèle.

9.4 Corrigé

1. **B.** Le routeur n'active que quelques experts, mais tous les poids restent en mémoire.
2. **C.** Le reranker re-note les candidats après une première recherche rapide.
3. **C.** Le RAG apporte du savoir récent avec sources, sans réentraînement.
4. **D.** QLoRA = LoRA sur modèle quantifié en 4 bits.
5. **A.** DPO apprend directement de paires préférée/rejetée.
6. **B.** C'est le catastrophic forgetting.
7. **C.** Pensée, action, observation, puis on recommence.
8. **D.** L'instruction malveillante vient d'un contenu récupéré, pas de l'utilisateur.
9. **B.** Une action à risque exige une validation humaine.
10. **C.** Le multi-agent se justifie par des domaines distincts, trop d'outils, du parallélisme ou des permissions séparées ; il consomme plus de tokens, pas moins.
11. **A.** Le lineage retrace le parcours et les transformations de la donnée.
12. **B.** Les journaux de production contiennent souvent des PII et du feedback biaisé.
13. **C.** Le streaming traite un flux continu avec faible latence.
14. **D.** Le prefill traite le prompt et détermine le TTFT.
15. **C.** Le prefix caching réutilise le KV cache d'un préfixe commun.
16. **B.** C'est le principe du speculative decoding.

17. **A.** Un lot plus grand augmente le débit total mais allonge chaque requête.
18. **B.** NIM = modèle + moteur optimisé + API dans un conteneur.
19. **C.** 70 milliards × 2 octets ≈ 140 Go.
20. **B.** MTTR = temps moyen de résolution ou de rétablissement (MTTD = détection).
21. **A.** Entraînement, simulation, calcul embarqué.
22. **B.** Randomisation, simulation plus fidèle et données réelles réduisent l'écart.

Comment lire votre score. Si vous ratez plusieurs questions d'un même chapitre, relisez-le et refaites ses exercices. Complétez ensuite avec les questions officielles de la page de l'examen, et vérifiez dans la documentation NVIDIA les outils récents (Agent Toolkit, NemoClaw, Switchyard, Dynamo) dont ce cours ne donne que le rôle général.